

ROS2 Spring Workshop

Bonus Lab: Bash Automation & The Art of Being Lazy

Project Context: Strategic Laziness

A good robotics engineer is a strategically lazy one. If you type a command more than three times a day, you should automate it. Typing `source install/setup.bash` in every new terminal is a rite of passage, but it's time to graduate.

Today, you will learn how to write a Bash script to automate your workflow, and you will encounter the classic Linux **Subshell Trap**.

Objective

To create a custom terminal command that automatically navigates to your workspace, builds your packages, and sources the environment all in one keystroke.

You will learn:

- **Bash Scripts:** How to write and execute a shell script.
- **Permissions:** Why Linux blocks files from running by default.
- **The `.bashrc` file:** How to configure your terminal's brain.
- **Aliases:** The professional way to create custom commands.

Part 1: Writing the Script (The Trap)

Let's write a script to do the heavy lifting. We want a script that changes directories, builds the code, and sources the environment.

1. **Create the file:** Navigate to your home directory and create a new file called `make_ros.sh`.

```
1 cd ~
2 code make_ros.sh
3
```

2. **Add the logic:** Open `make_ros.sh` in your code editor and add the following:

```
1 #!/bin/bash
2 echo "Building AgTech Workspace..."
3 cd /workspaces/agtech_ros2
4 colcon build --symlink-install
5
6 echo "Sourcing environment..."
7 source install/setup.bash
8
9 echo "Ready"
10
```

(Note: The `#!/bin/bash` at the top is called a “shebang.” It tells Linux exactly which interpreter to use to read the file).

3. **Make it executable:** By default, Linux assumes all new files are just text. For security, it will not allow you to run a text file as a program. You must explicitly grant it “execute” permissions.

```
1 chmod +x make_ros.sh
2
```

4. **Run it:** Execute the script using `./` (which means “run this file located in my current directory”).

```
1 ./make_ros.sh
2
```

Watch the terminal. It should successfully build the workspace and print your success messages.

5. **The Reality Check:** Now, try to run the node you built in Lab 3:

```
1 ros2 run lab3_safety_brake safety_stop
2
```

It will fail. ROS will complain that the package doesn't exist. But didn't the script *just* say it sourced the environment?

Critical Analysis: The Subshell Trap

When you run a script using `./make_ros.sh`, your terminal doesn't run the code itself. It creates a temporary "child" terminal (a subshell), runs the code inside it, and then instantly destroys the child when the script finishes.

The child successfully sourced the `setup.bash` file, but then it died, taking its environment variables with it! Your "parent" terminal remains completely unchanged.

The Takeaway: You cannot use a standard executable bash script to change the environment of your current terminal.

Part 2: The `.bashrc` File (The Real Solution)

To fix the Subshell Trap, we need to inject our commands directly into the terminal's permanent memory. Every time you open a new terminal window, it silently runs a hidden script called `.bashrc` before giving you control.

We are going to add an **Alias** to this file. An alias is a custom shortcut that runs directly in your current shell.

1. **Open your `.bashrc` file:** The tilde (`~`) represents your home directory.

```
1 code ~/.bashrc
2
```

2. **Create your Aliases:** Scroll to the very bottom of the file. Add these two lines:

```
1 # AgTech ROS 2 Shortcuts
2 alias src="source /workspaces/agtech_ros2/install/setup.bash && echo 'Workspace
   Sourced!'"
3 alias cb="cd /workspaces/agtech_ros2 && colcon build --symlink-install && src"
4
```

Let's break down what you just did:

- You created a command called `src` that instantly sources your workspace from anywhere.
- You created a master command called `cb` (colcon build). When you type `cb`, it chains three commands together using `&&`: it jumps to the workspace, builds it, and then runs your custom `src` command.

3. **Reload the terminal:** Since the `.bashrc` file only runs when a terminal *opens*, your current terminal doesn't know about the aliases yet. Tell it to re-read the file:

```
1 source ~/.bashrc
2
```

Part 3: The Test Drive

Your days of typing long setup commands are over.

1. Open a brand new terminal.

2. Type `cb` and hit enter.
3. Watch the terminal build the workspace and source it instantly.
4. Run your robot:

```
1 ros2 run lab3_safety_brake safety_stop  
2
```

Success! The node will run perfectly. You have successfully bypassed the subshell trap and automated your ROS 2 development cycle.